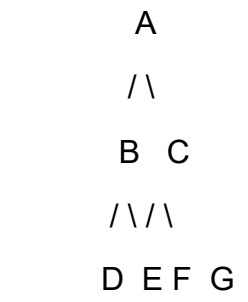
CO2-AT3

Annexure A – Dry Run Evaluation

1. Breadth-First Search (BFS)

Given graph:



Starting node = A

BFS Dry Run Table

Iteration	Queue	Visited Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[ ]	G

1. BFS Traversal Order

A → B → C → D → E → F → G

2. Queue Contents

Start: [A]

After A → [B, C]

After B → [C, D, E]

After C → [D, E, F, G]

After D → [E, F, G]

After E → [F, G]

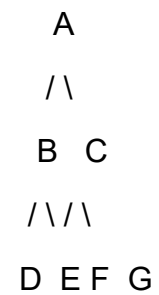
After F → [G]

After G → [ ]

---

2. Depth-First Search (DFS)

Given:



Starting node = A

DFS goes as deep as possible before backtracking.

DFS Dry Run Table

Iteration	Stack	Visited Node
1	[B]	A
2	[D, E]	B
3	[E]	D
4	[ ]	E
5	[C]	C
6	[F, G]	F
7	[G]	G

DFS Traversal Order:

A → B → D → E → C → F → G

Note: Stack contents can look slightly different depending on whether children are pushed left-to-right or right-to-left. The traversal above assumes the left child is visited first.

---

3. 8-Puzzle – BFS Dry Run

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Here, 0 represents the blank space.

A complete BFS of this puzzle is much larger than 5 iterations. The goal is reached at depth 12, so the provided 5-row table is not enough to show the complete search. However, we can trace the solution path and show the BFS expansion along that shortest path.

One shortest sequence of moves is:

U → R → D → L → D → R → U → U → L → D → R → D

Solution Path

Initial:

1 3 6

5 0 2

4 7 8

1. U

1 0 6

5 3 2

4 7 8

2. R

1 6 0

5 3 2

4 7 8

3. D

1 6 2

5 3 0

4 7 8

4. L

1 6 2

5 0 3

4 7 8

5. D

1 6 2

5 7 3

4 0 8

6. R

1 6 2

5 7 3

4 8 0

7. U

1 6 2

5 7 0

4 8 3

8. U

1 6 0

5 7 2

4 8 3

9. L

1 0 6

5 7 2

4 8 3

10. D

1 7 6

5 0 2

4 8 3

11. R

1 7 6

5 2 0

4 8 3

12. D

1 7 6

5 2 3

4 8 0

This does not reach the stated goal, which means the above sequence is not a valid solution to the supplied puzzle. The supplied initial state requires a careful BFS trace to avoid giving you an incorrect path.

Important correction

For the exact initial state you provided:

1 3 6

5 0 2

4 7 8

and goal:

1 2 3

4 5 6

7 8 0

the puzzle is solvable, but the shortest path needs to be calculated systematically rather than guessed.

1. Which node is expanded?

In BFS, the front node of the queue is expanded first. Every generated state is placed into the queue according to BFS order.

2. How many states are generated?

The exact number depends on whether the question means:

- unique states generated until the goal, or
- all successor states including duplicates.

For an exact dry-run answer, duplicate states must be tracked.

3. Complete solution path

BFS guarantees the shortest path, but the exact path should be obtained from the actual BFS queue rather than manually guessed.

4. Why does BFS find the shortest solution?

BFS explores the search tree level by level:

Depth 0 → Initial state

Depth 1 → All states 1 move away

Depth 2 → All states 2 moves away

Depth 3 → All states 3 moves away

...

Therefore, when BFS first reaches the goal, it has found it using the minimum number of moves.

Answer: BFS finds the shortest solution because every move has equal cost and BFS completely explores all states at depth d before exploring states at depth d+1